

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

COMPUTER ARCHITECTUUR

Labo 3

Tano Pannekoucke (r0955339)

Academic year 2025–2026

Inhoudsopgave

1 Inleiding **2**

2 Theoretische achtergrond **2**

3 Implementatie en Validatie **2**
3.1 Visuele Validatie 2

4 Resultaten en bespreking **3**
4.1 Algemene observaties 3
4.2 Diepere interpretatie 3

5 Conclusie **4**

A Code **6**

1 Inleiding

In dit labo wordt de invloed van **warp-organisatie**, **blockgrootte** en **thread divergence** op de performantie van CUDA-kernels onderzocht. We gebruiken een RGB-afbeelding (`anna.ppm`) en voeren drie kernels uit:

1. **Part 1 – Inversie:** eenvoudige kleurinversie ($x' = 255 - x$);
2. **Part 2A – Interleaved + branch:** een 1D-Gaussiaanse filter op het rode kanaal, inversie op G en B, met één kernel die vertakkingen bevat;
3. **Part 2B – Planar + geen branch:** dezelfde operatie, maar uitgevoerd op drie afzonderlijke planar-buffers (R, G, B) zonder vertakkingen.

Het doel is te begrijpen hoe:

- de blockgrootte (threads per block, TPB) de *warp-efficiëntie* beïnvloedt;
- conditionele takken binnen één warp leiden tot *divergence*;
- de keuze van datalayout (interleaved vs. planar) de performantie beïnvloedt.

2 Theoretische achtergrond

De NVIDIA GPU-architectuur maakt gebruik van een SIMT (Single-Instruction, Multiple-Thread) architectuur om duizenden threads efficiënt te beheren [1].

- **Warps:** Threads worden beheerd en uitgevoerd in groepen van 32, genaamd warps [1]. Alle threads in een warp starten op hetzelfde instructieadres.
- **Volle vs. Partiële Warps:** Het aantal warps in een blok wordt berekend als $\lceil T/32 \rceil$. Als het aantal threads T geen veelvoud is van 32, bevat de laatste warp inactieve threads die wel resources innemen maar geen werk verrichten [1].
- **Warp divergence:** Threads binnen een warp kunnen onafhankelijk vertakken. Echter, als ze divergeren (bijv. via een 'if-else'), voert de warp beide paden sequentieel uit, waarbij threads die niet op het huidige pad zitten tijdelijk worden uitgeschakeld. Dit verlaagt de efficiëntie aanzienlijk [1].

Ook het **geheugen-toegangspatroon** is belangrijk:

- **Interleaved** RGB (RGBRGB...): kan inefficiënt zijn als slechts één kanaal complex bewerkt wordt.
- **Planar** (RRR...GGG...BBB...): maakt opeenvolgende, goed gecoalesceerde toegang per kanaal mogelijk.

3 Implementatie en Validatie

Alle kernels gebruiken een *grid-stride loop*:

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;
int stride = blockDim.x * gridDim.x;
for (int i = tid; i < total; i += stride) { ... }
```

Part 1 voert enkel een inversie uit ($x' = 255 - x$) op alle kleurcomponenten.

Part 2A werkt op interleaved data in één kernel. Het R-kanaal krijgt een 1D Gaussiaanse filter, terwijl G/B geïnverteerd worden. Dit veroorzaakt warp divergence.

Part 2B zet de data eerst om naar planar buffers (R, G, B) en gebruikt aparte kernels. Hier voeren alle threads binnen een warp dezelfde instructies uit (geen divergence).

3.1 Visuele Validatie

Om aan te tonen dat de logica correct werkt, is de output visueel geïnspecteerd (zie Figuur 1). De rode waas in de output van Part 2A/B bevestigt dat het rode kanaal geblurred (behoudt hoge waarden) is en de andere kanalen geïnverteerd zijn.



(a) Origineel (b) Part 1: Inversie (c) Part 2: R-Blur + Inv

Figuur 1: Visuele validatie van de kernels. In (c) is te zien hoe de lichte vacht rood kleurt door de bewerking.

4 Resultaten en bespreking

De uitvoeringstijd werd gemeten over 100 runs (na warm-up). De resultaten zijn samengevat in Tabel 1 en Figuur 2.

4.1 Algemene observaties

Voor alle varianten vertonen de resultaten een U-vormig patroon (Figuur 2):

- **Links:** kleine TPB → veel blocks, hoge scheduler-overhead.
- **Rechts:** zeer grote TPB → minder blocks per SM, lagere occupancy.
- **Midden:** beste resultaten rond 3–4 volle warps per block (TPB 96-128).

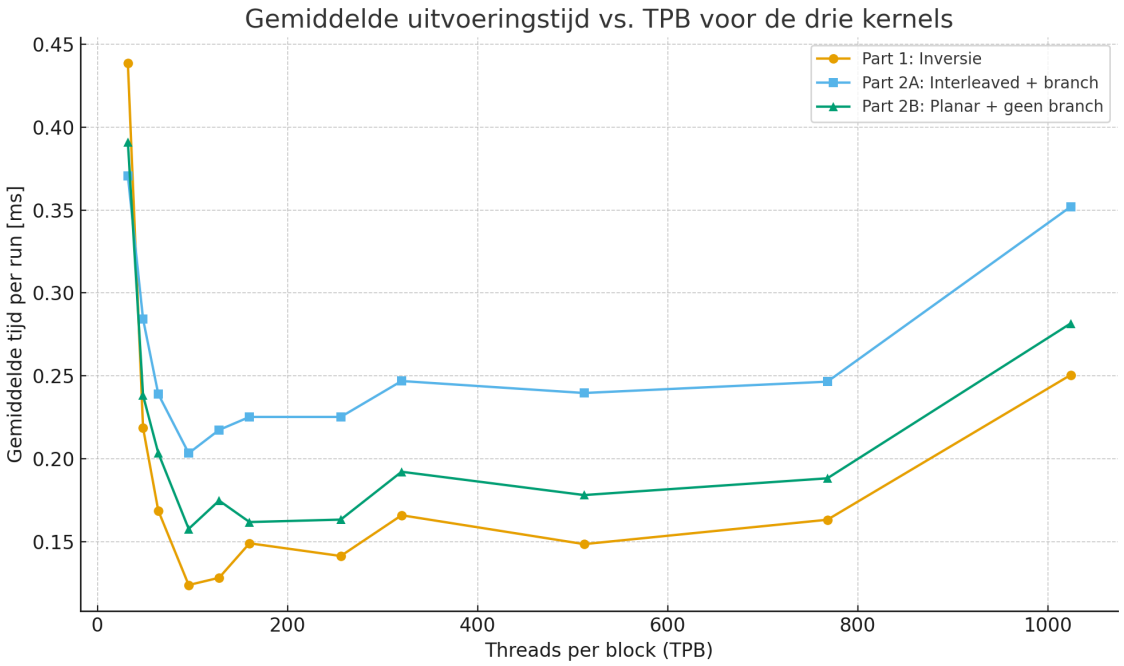
4.2 Diepere interpretatie

Warm-up effect. Zoals te zien in Figuur 3, is de eerste run (index 1) aanzienlijk trager dan de daaropvolgende runs. Dit valideert het gebruik van een 'warm-up' run in de meetmethode.

Kleine TPB's (32–48). Veel kleine blocks leiden tot relatief hoge overhead. Bij TPB=48 zijn bovendien partiële warps actief (48 threads = 1.5 warps), wat betekent dat 50% van de resources van de tweede warp verspild worden [1].

Gemiddelde TPB's (96–128). Hier vallen de minima voor alle kernels. Er is een goede balans: volle warps en voldoende blocks per SM om latency te verbergen.

Invloed van divergence (Part 2A vs 2B). Bij de optimale configuratie (TPB=96) is Part 2B (≈ 0.158 ms) ongeveer **22% sneller** dan Part 2A (≈ 0.203 ms). In Part 2A dwingt de 'if-else' structuur de warp om beide paden sequentieel uit te voeren (divergence). Part 2B splitst dit op in aparte kernels, waardoor alle threads in een warp altijd hetzelfde doen. Daarnaast profiteert Part 2B van *coalesced memory access* door de planar layout.



Figuur 2: Gemiddelde uitvoeringstijd per kernelrun als functie van TPB. Merk de sterke daling op bij het bereiken van volle warps.

5 Conclusie

De metingen tonen duidelijk aan dat GPU-performantie afhangt van:

1. **Warp-efficiëntie:** Gebruik veelvouden van 32 (TPB 96-128 was optimaal) om partiële warps te vermijden.
2. **Uniforme uitvoering:** Vermijd divergence door taken op te splitsen (Part 2B was significant sneller dan 2A).
3. **Geheugenlayout:** Planar layouts verhogen de efficiëntie door gecoalesceerde toegang.

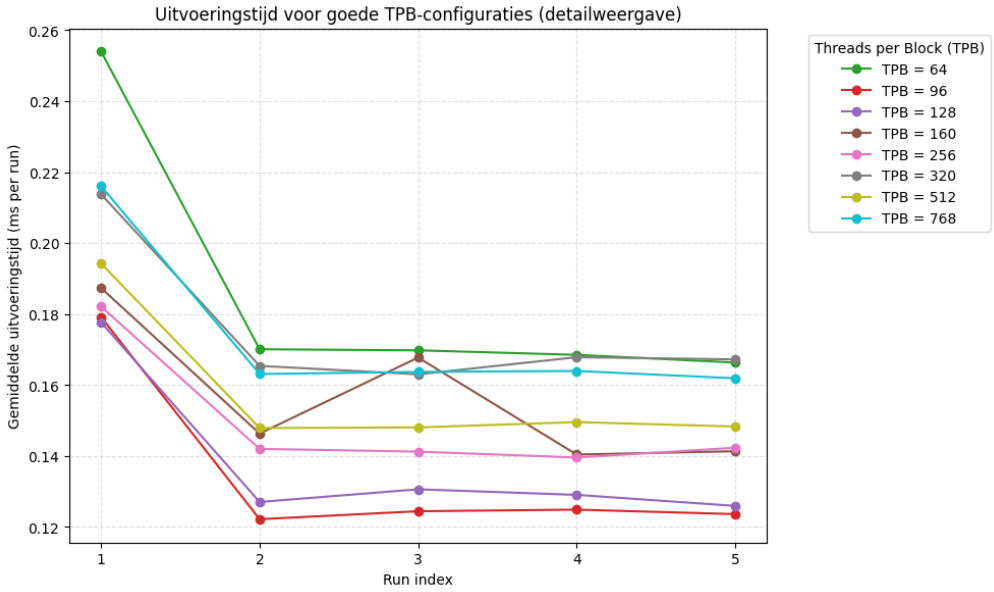
Tabel 1: Gemiddelde uitvoeringstijd per kernelrun (100 runs, `anna.ppm`), geclusterd per TPB.

TPB	Warps/block	Volle warps	Gem. tijd [ms/run]	Opmerking
		Part 1 – Inversie		
32	1	ja	≈ 0.44	Veel kleine blocks, scheduler-overhead
48	2	nee	≈ 0.22	Partiële warps
64	2	ja	≈ 0.17	Volle warps, beter
96	**3**	**ja**	$\approx$ **0.124**	**Optimaal**
128	**4**	**ja**	$\approx$ **0.128**	**Quasi optimaal**
160	5	ja	≈ 0.149	Volle warps, minder gunstige scheduling
256	8	ja	≈ 0.141	Goed, maar minder flexibel
320	10	ja	≈ 0.166	Volle warps, minder blocks/SM
512	16	ja	≈ 0.148	Lagere occupancy
768	24	ja	≈ 0.163	Idem
1024	32	ja	≈ 0.250	Te weinig blocks per SM
		Part 2A – Interleaved + branch (divergent)		
32	1	ja	≈ 0.37	Kleine blocks + divergence
64	2	ja	≈ 0.24	
96	**3**	**ja**	$\approx$ **0.203**	**Beste zone**
128	4	ja	≈ 0.217	Nog goed
160–320			≈ 0.225–0.248	Iets trager
512–768			≈ 0.24–0.25	Lagere occupancy
1024	32	ja	≈ 0.35	Te weinig parallelisme
		Part 2B – Planar + geen branch		
32	1	ja	≈ 0.39	Te kleine blocks, overhead
64	2	ja	≈ 0.20	
96	**3**	**ja**	$\approx$ **0.158**	**Beste prestatie**
128	4	ja	≈ 0.175	Iets trager
160–256			≈ 0.162–0.163	Quasi optimaal
320–768			≈ 0.19–0.19	Langzaam stijgend
1024	32	ja	≈ 0.28	Te weinig blocks per SM

Referenties

[1] NVIDIA Corporation. *CUDA C++ Programming Guide, Release 12.3*. Chapter 7: Hardware Implementation (SIMT Architecture, Hardware Multithreading).

[2] Google. (2025). *Gemini* . `https://gemini.google.com`



Figuur 3: Detailweergave van de eerste 5 runs. De vertraging bij run 1 toont het warm-up effect.

A Code

Hieronder bevindt zich de volledige C++/CUDA implementatie gebruikt voor dit labo.

```
/*
 * CUDA Image Processing     Part 1 + Part 2
 *
 * Part 1: Kleurinversie (al door jou gemaakt)
 * Part 2A: Interleaved ( R G B R G B ) met branch: R krijgt 1D Gaussian blur, G/B inverse
 * Part 2B: Planar ( R R R   G G G   B B B ) zonder branch: R blur, G & B inverse
 *
 */

#include <cstdint>      // Data types
#include <iostream>     // std::cout
#include <cstdio>       // FILE*, fopen, fread, fwrite
#include <cstdlib>      // exit
#include <cuda_runtime.h>

// #define M 512
// #define N 512
#define M 960       // width
#define N 1280      // height
#define C 3         // Colors (RGB)
#define OFFSET 16   // Header length (ruim genoeg voor P6 header)

// ----------------------- Utils / CUDA-check -----------------------
#define CUDA_CHECK(x) do {                                          \
  cudaError_t e = (x);                                              \
  if (e != cudaSuccess) {                                           \
    fprintf(stderr, "CUDA ERROR %s:%d: %s\n", __FILE__, __LINE__,   \
            cudaGetErrorString(e));                                 \
    std::exit(EXIT_FAILURE);                                        \
  }                                                                 \
} while(0)

__device__ __forceinline__ int clampi(int x, int lo, int hi){
    return x < lo ? lo : (x > hi ? hi : x);
}

// ----------------------- I/O helpers -----------------------
uint8_t* get_image_array(void){
    FILE *imageFile = fopen("./anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open input file");
```

```
            exit(EXIT_FAILURE);
        }
        // Allocate inclusief header-ruimte zodat we direct kunnen fread
        uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t) + OFFSET);
        if (!image_array) {
            perror("ERROR: malloc failed");
            exit(EXIT_FAILURE);
        }
        size_t want = M*N*C*sizeof(uint8_t) + OFFSET;
        size_t got  = fread(image_array, 1, want, imageFile);
        if (got < OFFSET) {
            fprintf(stderr, "ERROR: Unexpected file size.\n");
            exit(EXIT_FAILURE);
        }
        fclose(imageFile);
        return image_array + OFFSET; // pixeldata start
    }

    static void save_interleaved_as_ppm(const char* path, const uint8_t* interleaved){
        FILE *imageFile = fopen(path,"wb");
        if(imageFile==NULL){
            perror("ERROR: Cannot open output file");
            exit(EXIT_FAILURE);
        }
        // P6 header
        fprintf(imageFile,"P6\n");
        fprintf(imageFile,"%d %d\n", M, N);
        fprintf(imageFile,"255\n");
        // Write pixels
        fwrite(interleaved, 1, M*N*C, imageFile);
        fclose(imageFile);
    }

    // ----------------------- Part 1: invert (interleaved) -----------------------
    __global__ void invert_kernel(const uint8_t* __restrict__ in,
                                  uint8_t* __restrict__ out,
                                  int total) // total = M*N*C
    {
        int tid = blockIdx.x * blockDim.x + threadIdx.x;
        int stride = blockDim.x * gridDim.x;
        for (int i = tid; i < total; i += stride) {
            out[i] = 255u - in[i];
        }
    }

    // ----------------------- Part 2A: interleaved + branch -----------------------
    /*
     * Interleaved RGB: index i => chan = i % 3 (0=R,1=G,2=B), pixel p = i/3
     * R-kanaal: 1D Gaussian over pixelindex p (buren op p 1 , p 2),
     *           corresponderend met interleaved indices 3*p + chan(=0).
     * G/B: gewone invert.
     * Dit geeft divergence binnen warps (if(chan==0) ... else ...).
     */
    __global__ void invert_or_blur_R_interleaved(
        const uint8_t* __restrict__ in,
        uint8_t* __restrict__ out,
        int pixels) // pixels = M*N
    {
        int total = pixels * 3;
        int tid   = blockIdx.x * blockDim.x + threadIdx.x;
        int stride= blockDim.x * gridDim.x;

        for (int i = tid; i < total; i += stride) {
            int chan = i % 3;       // 0=R,1=G,2=B
            int p    = i / 3;       // pixel index [0..pixels-1]
            if (chan == 0) {
                // Gaussian 1D op R (alleen horizontale pixel-index p, geen 2D convolutie)
                int p_m2 = clampi(p - 2, 0, pixels-1);
                int p_m1 = clampi(p - 1, 0, pixels-1);
                int p_p1 = clampi(p + 1, 0, pixels-1);
                int p_p2 = clampi(p + 2, 0, pixels-1);

                int i_m2 = 3*p_m2 + 0;
```

```
                int i_m1 = 3*p_m1 + 0;
                int i_0  = 3*p    + 0;
                int i_p1 = 3*p_p1 + 0;
                int i_p2 = 3*p_p2 + 0;

                float v = 0.10f*in[i_m2] + 0.25f*in[i_m1] + 0.50f*in[i_0]
                        + 0.25f*in[i_p1] + 0.10f*in[i_p2];
                out[i] = (uint8_t)(v + 0.5f);
            } else {
                out[i] = 255u - in[i]; // G en B
            }
        }
    }

    // ----------------------- Part 2B: planar + no branch -----------------------
    __global__ void deinterleave(const uint8_t* in, uint8_t* R, uint8_t* G, uint8_t* B, int
    pixels){
        int tid = blockIdx.x * blockDim.x + threadIdx.x;
        int stride= blockDim.x * gridDim.x;
        for (int p = tid; p < pixels; p += stride){
            int base = 3*p;
            R[p] = in[base + 0];
            G[p] = in[base + 1];
            B[p] = in[base + 2];
        }
    }

    __global__ void interleave(const uint8_t* R, const uint8_t* G, const uint8_t* B, uint8_t*
     out, int pixels){
        int tid = blockIdx.x * blockDim.x + threadIdx.x;
        int stride= blockDim.x * gridDim.x;
        for (int p = tid; p < pixels; p += stride){
            int base = 3*p;
            out[base + 0] = R[p];
            out[base + 1] = G[p];
            out[base + 2] = B[p];
        }
    }

    __global__ void gauss1d_red(const uint8_t* Rin, uint8_t* Rout, int pixels){
        int tid = blockIdx.x * blockDim.x + threadIdx.x;
        int stride= blockDim.x * gridDim.x;
        for (int i = tid; i < pixels; i += stride){
            int i_m2 = clampi(i - 2, 0, pixels-1);
            int i_m1 = clampi(i - 1, 0, pixels-1);
            int i_p1 = clampi(i + 1, 0, pixels-1);
            int i_p2 = clampi(i + 2, 0, pixels-1);
            float v = 0.10f*Rin[i_m2] + 0.25f*Rin[i_m1] + 0.50f*Rin[i]
                    + 0.25f*Rin[i_p1] + 0.10f*Rin[i_p2];
            Rout[i] = (uint8_t)(v + 0.5f);
        }
    }

    __global__ void invert_planar(const uint8_t* in, uint8_t* out, int pixels){
        int tid = blockIdx.x * blockDim.x + threadIdx.x;
        int stride= blockDim.x * gridDim.x;
        for (int i = tid; i < pixels; i += stride){
            out[i] = 255u - in[i];
        }
    }

    // ----------------------- main -----------------------
    int main (void) {
        // ---------------- Part 1: lees en inverseer ----------------
        uint8_t* image_array = get_image_array(); // interleaved input
        const int total  = M * N * C;
        const int pixels = M * N;

        // Host output buffers
        uint8_t* out_part1  = (uint8_t*)malloc(total);
        uint8_t* out_part2A = (uint8_t*)malloc(total);
        uint8_t* out_part2B = (uint8_t*)malloc(total);
        if(!out_part1 || !out_part2A || !out_part2B){
```

```
            fprintf(stderr, "ERROR: host malloc failed\n");
            return 1;
        }

        // Device buffers (shared waar zinvol)
        uint8_t *dIn = nullptr, *dOut = nullptr;
        CUDA_CHECK(cudaMalloc((void**)&dIn,  total));
        CUDA_CHECK(cudaMalloc((void**)&dOut, total));
        CUDA_CHECK(cudaMemcpy(dIn, image_array, total, cudaMemcpyHostToDevice));

        // TPB sweep (behouden)
        int tpbList[] = { 32, 48, 64, 96, 128, 160, 256, 320, 512, 768, 1024 };
        const int numConfigs = sizeof(tpbList)/sizeof(tpbList[0]);
        const int iters = 100;

        std::cout << "=== Part 1: Invert (interleaved) ===\n";
        for (int k = 0; k < numConfigs; ++k) {
            int tpb = tpbList[k];
            int blocks = (total + tpb - 1) / tpb;

            invert_kernel<<<blocks, tpb>>>(dIn, dOut, total);
            CUDA_CHECK(cudaDeviceSynchronize());

            cudaEvent_t start, stop;
            CUDA_CHECK(cudaEventCreate(&start));
            CUDA_CHECK(cudaEventCreate(&stop));

            CUDA_CHECK(cudaEventRecord(start));
            for (int it = 0; it < iters; ++it) {
                invert_kernel<<<blocks, tpb>>>(dIn, dOut, total);
            }
            CUDA_CHECK(cudaEventRecord(stop));
            CUDA_CHECK(cudaEventSynchronize(stop));

            float ms = 0.f;
            CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
            std::cout << "TPB=" << tpb
                      << "  blocks=" << blocks
                      << "  time=" << ms << " ms for "
                      << iters << " runs  (avg = " << (ms/iters) << " ms)\n";

            CUDA_CHECK(cudaEventDestroy(start));
            CUDA_CHECK(cudaEventDestroy(stop));
        }
        CUDA_CHECK(cudaMemcpy(out_part1, dOut, total, cudaMemcpyDeviceToHost));
        save_interleaved_as_ppm("./output_image.ppm", out_part1);

        // ---------------- Part 2A: interleaved + branch (divergence) ----------------
        std::cout << "\n=== Part 2A: Interleaved + branch (R blur, G/B invert) ===\n";
        for (int k = 0; k < numConfigs; ++k) {
            int tpb = tpbList[k];
            int blocks = (total + tpb - 1) / tpb;

            // Warm-up
            invert_or_blur_R_interleaved<<<blocks, tpb>>>(dIn, dOut, pixels);
            CUDA_CHECK(cudaDeviceSynchronize());

            // Time alleen de kernel (geen H2D/D2H)
            cudaEvent_t start, stop;
            CUDA_CHECK(cudaEventCreate(&start));
            CUDA_CHECK(cudaEventCreate(&stop));

            CUDA_CHECK(cudaEventRecord(start));
            for (int it = 0; it < iters; ++it) {
                invert_or_blur_R_interleaved<<<blocks, tpb>>>(dIn, dOut, pixels);
            }
            CUDA_CHECK(cudaEventRecord(stop));
            CUDA_CHECK(cudaEventSynchronize(stop));

            float ms = 0.f;
            CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
            std::cout << "TPB=" << tpb
                      << "  blocks=" << blocks
```

```
                      << "  time=" << ms << " ms for "
                      << iters << " runs  (avg = " << (ms/iters) << " ms)\n";

            CUDA_CHECK(cudaEventDestroy(start));
            CUDA_CHECK(cudaEventDestroy(stop));
        }
        CUDA_CHECK(cudaMemcpy(out_part2A, dOut, total, cudaMemcpyDeviceToHost));
        save_interleaved_as_ppm("./output_part2A.ppm", out_part2A);

        // ---------------- Part 2B: planar + no branch (geen divergence) ----------------
        std::cout << "\n=== Part 2B: Planar (no branch) ===\n";
        // Allocate planar device buffers
        uint8_t *dR=nullptr, *dG=nullptr, *dB=nullptr;
        uint8_t *dRout=nullptr, *dGout=nullptr, *dBout=nullptr;
        CUDA_CHECK(cudaMalloc((void**)&dR, pixels));
        CUDA_CHECK(cudaMalloc((void**)&dG, pixels));
        CUDA_CHECK(cudaMalloc((void**)&dB, pixels));
        CUDA_CHECK(cudaMalloc((void**)&dRout, pixels));
        CUDA_CHECK(cudaMalloc((void**)&dGout, pixels));
        CUDA_CHECK(cudaMalloc((void**)&dBout, pixels));

        // Deinterleave (  nmalig   buiten tijdmeting, kan je ook meenemen als je wil)
        {
            int tpb = 256;
            int blocks = (pixels + tpb - 1) / tpb;
            deinterleave<<<blocks, tpb>>>(dIn, dR, dG, dB, pixels);
            CUDA_CHECK(cudaDeviceSynchronize());
        }

        // Sweep TPB en meet alleen de 3 kernels (R blur, G inv, B inv)
        for (int k = 0; k < numConfigs; ++k) {
            int tpb = tpbList[k];
            int blocks = (pixels + tpb - 1) / tpb;

            // Warm-up
            gauss1d_red<<<blocks, tpb>>>(dR, dRout, pixels);
            invert_planar<<<blocks, tpb>>>(dG, dGout, pixels);
            invert_planar<<<blocks, tpb>>>(dB, dBout, pixels);
            CUDA_CHECK(cudaDeviceSynchronize());

            cudaEvent_t start, stop;
            CUDA_CHECK(cudaEventCreate(&start));
            CUDA_CHECK(cudaEventCreate(&stop));

            CUDA_CHECK(cudaEventRecord(start));
            for (int it = 0; it < iters; ++it) {
                gauss1d_red<<<blocks, tpb>>>(dR, dRout, pixels);
                invert_planar<<<blocks, tpb>>>(dG, dGout, pixels);
                invert_planar<<<blocks, tpb>>>(dB, dBout, pixels);
            }
            CUDA_CHECK(cudaEventRecord(stop));
            CUDA_CHECK(cudaEventSynchronize(stop));

            float ms = 0.f;
            CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
            std::cout << "TPB=" << tpb
                      << "  blocks=" << blocks
                      << "  time=" << ms << " ms for "
                      << iters << " runs  (avg = " << (ms/iters) << " ms)\n";

            CUDA_CHECK(cudaEventDestroy(start));
            CUDA_CHECK(cudaEventDestroy(stop));
        }

        // Interleave (  nmalig   buiten tijdmeting, of neem mee indien gewenst)
        {
            int tpb = 256;
            int blocks = (pixels + tpb - 1) / tpb;
            interleave<<<blocks, tpb>>>(dRout, dGout, dBout, dOut, pixels);
            CUDA_CHECK(cudaDeviceSynchronize());
        }
        CUDA_CHECK(cudaMemcpy(out_part2B, dOut, total, cudaMemcpyDeviceToHost));
        save_interleaved_as_ppm("./output_part2B.ppm", out_part2B);
```

```
332
333         // ---------------- Cleanup ----------------
334         CUDA_CHECK(cudaFree(dBout));
335         CUDA_CHECK(cudaFree(dGout));
336         CUDA_CHECK(cudaFree(dRout));
337         CUDA_CHECK(cudaFree(dB));
338         CUDA_CHECK(cudaFree(dG));
339         CUDA_CHECK(cudaFree(dR));
340         CUDA_CHECK(cudaFree(dOut));
341         CUDA_CHECK(cudaFree(dIn));
342
343         free(out_part2B);
344         free(out_part2A);
345         free(out_part1);
346         // image_array wijst in het midden van een alloc (OFFSET), laten we ongemoeid.
347
348         return 0;
349     }
```